



Carrera:

**LICENCIATURA EN PROGRAMACIÓN
Y WEB MÁSTER**

Docente:

Mario Alonso Segovia Gutiérrez

Materia:

PROGRAMACIÓN AVANZADA

Alumno(s):

José Gabriel Montero Lozano

Fecha:

04/08/2026

Patrones de diseño de software

Patrón	Categoría	Problema que resuelve	Objetivo principal	Funcionamiento general	Ventajas	Desventajas	Casos de uso	Ejemplo de aplicación	Complejidad
Singleton	Creacional	Evita múltiples instancias y acceso descontrolado a un recurso compartido	Garantizar una única instancia global	Constructor privado, instancia estática, método de acceso único	Control central, menor uso de memoria	Puede introducir acoplamiento y dificultar pruebas	Configuración, logging, caché, gestores globales	Gestor de configuración de una app de escritorio o servicio compartido	Baja-media
Factory Method	Creacional	El cliente no debe decidir la clase concreta a instanciar	Delegar la creación a subclases o fábricas	Define una interfaz de creación y deja la decisión al subtipado	Menor acoplamiento, más extensibilidad	Más clases y abstracción adicional	Creación de documentos, notificaciones, exportadores	Sistema de exportación que crea PDF, CSV o Excel según el tipo	Media
Abstract Factory	Creacional	Debe crear familias de objetos compatibles entre sí	Proveer interfaces para crear familias relacionadas	Una fábrica produce múltiples productos coherentes	Consistencia entre objetos, fácil cambiar de familia	Complejidad y más código de soporte	UI multiplataforma, temas, suites de componentes	Kit de interfaz que crea botones, menús y ventanas para Windows o macOS	Media-alta
Builder	Creacional	Objetos complejos con muchas combinaciones de construcción	Separar construcción paso a paso de la representación	Un director coordina pasos y el builder arma el objeto	Flexibilidad, claridad, evita constructores enormes	Más clases y sobreingeniería si el objeto es simple	Queries, reportes, configuraciones, APIs fluent	Generador de reportes que arma un PDF con secciones opcionales	Media
Adapter	Estructural	Interfaces incompatibles entre sí	Hacer que clases incompatibles colaboren	Envuelve una interfaz y traduce llamadas	Reutiliza código existente, integra sistemas legados	Puede añadir una capa extra y complejidad	Integración de APIs, librerías antiguas, migraciones	Adaptar una API externa de pagos a la interfaz interna de la app	Baja-media
Facade	Estructural	Subsistemas complejos difíciles de usar	Ofrecer una interfaz simple de alto nivel	Un objeto fachada coordina múltiples subsistemas	Reduce complejidad, mejora legibilidad	Puede ocultar detalles importantes	Orquestación de servicios, capas de infraestructura, SDKs	Fachada para iniciar sesión, cargar perfil y permisos en una app	Baja
Decorator	Estructural	Agregar responsabilidades sin crear subclases combinatorias	Extender comportamiento dinámicamente	Envoltorios encadenables añaden funciones	Muy flexible, evita explosión de herencia	Más objetos y flujo más difícil de seguir	Streams, UI, logging, validaciones, middleware	Añadir compresión y cifrado a un canal de envío de archivos	Media
Observer	Comportamiento	Varias partes deben reaccionar a cambios de estado	Notificar automáticamente a suscriptores	El sujeto publica eventos y los observadores se actualizan	Bajo acoplamiento, buena escalabilidad	Riesgo de cascadas de eventos y difícil trazabilidad	UI reactiva, eventos de dominio, notificaciones	Panel de métricas que actualiza widgets al cambiar los datos	Media
Strategy	Comportamiento	Varias formas de resolver una misma tarea	Intercambiar algoritmos sin cambiar al cliente	El contexto delega en una estrategia seleccionable	Código más limpio, fácil de probar	Más objetos y configuración adicional	Reglas de descuento, ordenamiento, validación, rutas	Motor de precios que cambia entre estrategias de descuento	Baja-media
Command	Comportamiento	Encapsular acciones, colas o deshacer/rehacer	Convertir una solicitud en un objeto	Un comando guarda la acción y su receptor	Permite undo/redo, colas y registro	Más clases y verbosidad	Edición, automatización, macros, tareas diferidas	Editor que guarda acciones para deshacer y rehacer cambios	Media

REFERENCIAS:

R <https://refactorin.guru/es/design-patterns>
<https://refactoring.guru/es/design-patterns/singleton>
<https://refactoring.guru/es/design-patterns/factory-method>
<https://refactoring.guru/es/design-patterns/abstract-factory>
<https://refactoring.guru/es/design-patterns/builder>
<https://refactoring.guru/es/design-patterns/adapter>
<https://refactoring.guru/es/design-patterns/facade>
<https://refactoring.guru/es/design-patterns/decorator>
<https://refactoring.guru/es/design-patterns/observer>
<https://refactoring.guru/es/design-patterns/strategy>
<https://refactoring.guru/es/design-patterns/command>